Core Building Blocks for Low Latency Applications in C++

Flynn Yuying Guo

Based on Chapter 4 of "Building Low Latency Applications with C++" by Sourav Ghosh

Agenda

1. Thread Utilities
2. Memory Pool
3. Lock-Free Queue
4. Low-Latency Logging Framework
5. TCP/UDP Network Sockets

Why we need five building blocks?

In a low-latency trading system, every microsecond counts.These five components form the foundation that ALL higher-level systems (matching engine, market data, order gateway) are built upon.

Problems:

- Dynamic memory allocation (new/delete) is too slow — 100s~1000s ns per call
- Locks (mutex) cause context switches — 1~10 μs overhead
- Synchronous I/O blocks the critical path — disk writes take μs to ms
- Standard networking adds kernel overhead

Solutions:

- Pre-allocate all memory at startup (Memory Pool)
- Use lock-free data structures (LFQueue) instead of mutexes
- Offload I/O to background threads (Async Logger)
- Pin threads to dedicated CPU cores (Thread Affinity)
- Use non-blocking sockets with epoll

1.
Thread Utilities

Creating threads, pinning them to CPU cores, and foundational macros

Files: macros.h, thread_utils.h, thread_example.cpp

Branch Prediction Hints — LIKELY / UNLIKELY

CPU pipelines speculate which branch an if-statement will take. A wrong guess flushes the pipeline (10-20 cycle penalty). We give the compiler hints to arrange code optimally.

```
#define LIKELY(x)   __builtin_expect(!!(x), 1)
#define UNLIKELY(x) __builtin_expect(!!(x), 0)

// Usage:
if (LIKELY(order != nullptr)) { process(order); }   // fast path
if (UNLIKELY(error)) { handle_error(); }             // rare
```

How it works:

The compiler places the likely-taken branch in the fall-through code path (no jump needed), and the unlikely branch further away (requires a jump). This aligns with how CPU instruction caches and pre-fetchers work.

ASSERT and FATAL — Fail-Fast Error Handling

```
inline auto ASSERT(bool cond, const std::string& msg) noexcept {
    if (UNLIKELY(!cond)) {          // Condition almost never
fails   std::cerr << msg << std::endl;
        exit(EXIT_FAILURE);         // Crash immediately
    }
}

inline auto FATAL(const std::string& msg) noexcept {
    std::cerr << msg << std::endl;
    exit(EXIT_FAILURE);             // Unconditional crash
}
```

Key design decisions:

- Uses UNLIKELY because assertion failures are exceptional (bugs only)
- noexcept — no exception overhead; we crash deliberately instead

createAndStartThread — Core Thread Creation

```cpp
template<typename T, typename... A>
inline auto createAndStartThread(int core_id,
                                 const std::string &name,
                                 T &&func, A &&... args) noexcept {
    std::atomic<bool> running(false), failed(false);

    auto thread_body = [&] {
        if (core_id >= 0 && !setThreadCore(core_id))
            { failed = true; return; }
        running = true;
        std::forward<T>(func)(std::forward<A>(args)...);
    };
    auto t = new std::thread(thread_body);
    while (!running && !failed) std::this_thread::sleep_for(1s);
    if (failed) { t->join(); delete t; t = nullptr; }
    return t;
}
```

Variadic templates (typename... A) accept any function signature

std::forward — perfect forwarding avoids unnecessary copies

std::atomic<bool> for safe cross-thread communication (no mutex needed)

Busy-wait loop ensures affinity is set before the function returns

setThreadCore — CPU Core Affinity

Each CPU core has its own L1/L2 cache. If a thread migrates between cores, cached data is invalidated — costing 100+ nanoseconds per cache miss.

```
inline auto setThreadCore(int core_id) noexcept {
    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);              // Clear all bits
    CPU_SET(core_id, &cpuset);      // Set target core
    return (pthread_setaffinity_np(
        pthread_self(),
        sizeof(cpu_set_t),
        &cpuset) == 0);
}
```

Three Steps to Pin:

1. CPU_ZERO → [0,0,0,0,...]
2. CPU_SET(2)→ [0,0,1,0,...]
3. pthread_setaffinity_np()

Thread Example — Execution Flow

```
auto t1 = createAndStartThread(
    -1, "func1",        // no core pin
    dummyFunction, 12, 21, false);

auto t2 = createAndStartThread(
    1, "func2",         // pin to core 1
    dummyFunction, 15, 51, true);
```

Timeline:

Main → creates t1 → creates t2

t1 → runs fast, finishes

t2 → runs, sleeps 5s, finishes

Main → join(t1), join(t2), exit

Thread Creation — Detailed Execution Flow

How createAndStartThread() coordinates main thread and new thread:

```
Main Thread                                New Thread
    │                                          │
    │── new std::thread(body) ────────────>    │
    │                                          │
    │                                          ├─ setThreadCore(core_id)
    │                                          │     ├─ CPU_ZERO
    │    [busy-wait loop]                      │     ├─ CPU_SET
    │    while(!running && !failed)            │     └─ pthread_setaffinity_np
    │        sleep_for(1s)                     │
    │                                          ├─ if failed → failed=true; return
    │                                          │
    │                                          ├─ running = true ──────────┐
    │                                          │                           │
    │<──── running detected ──────────────┘                                │
    │                                                                      │
    │                                          ├─ forward<T>(func)(args...)
    │── return thread pointer                  │     └─ user function runs
    │── continue main logic                    │
    │                                          │── function completes
    │── t->join() ──────────────────────>      │── thread exits
    ▼                                          ▼
```

2.
Memory Pool

Pre-allocated memory management to eliminate runtime dynamic allocation

Files: mem_pool.h, mem_pool_example.cpp

Why Avoid Dynamic Allocation?

100-5000

nanoseconds per new/delete call

5-50

nanoseconds per pool allocate

10-100×

speedup with memory pools

Golden Rule:

In a low-latency trading system, all memory must be pre-allocated before the critical path begins.

Memory Pool — Data Structure Design

```
template<typename T> class MemPool final {
    struct ObjectBlock { T object_; bool is_free_ = true; };  // Packed together
    std::vector<ObjectBlock> store_;     // Pre-allocated at construction
    size_t next_free_index_ = 0;         // Points to next available slot
};
```

Packed ObjectBlock:

- struct ObjectBlock { T object_; bool is_free_; }
- Object and its flag are adjacent in memory
- One cache line load gets both object + flag
- vector<ObjectBlock> — single contiguous array

Memory Pool — allocate() with Placement New

```
template<typename... Args>
T* allocate(Args... args) noexcept {
    auto obj_block = &(store_[next_free_index_]);
    ASSERT(obj_block->is_free_, "Expected free block");

    T* ret = &(obj_block->object_);
    ret = new(ret) T(args...);    // placement new — construct in-
place!
    obj_block->is_free_ = false;  // mark as used
    updateNextFreeIndex();        // find next free slot
    return ret;
}
```

Placement new:

new(address) T(args...) — constructs the object at a pre-existing memory address. No memory allocation happens; it only calls the constructor.

updateNextFreeIndex:

Scans forward from current index, wraps around like a circular buffer. If it returns to the starting position without finding a free slot → pool is full (ASSERT fails).

Memory Pool — deallocate() and Safety

```cpp
auto deallocate(const T* elem) noexcept {
    const auto idx = (
      reinterpret_cast<const ObjectBlock*>(elem)
      - &store_[0]);   // pointer arithmetic

    ASSERT(idx >= 0 && idx < store_.size(), ...);
    ASSERT(!store_[idx].is_free_, ...);  // double-free?
    store_[idx].is_free_ = true;  // That's it!
}
```

Why reinterpret_cast works:

object_ is the FIRST member of ObjectBlock, so its address equals the struct's address.

Pointer subtraction gives the array index — no search needed!

Safety features built into MemPool:

- = delete on copy/move constructors — prevents accidental duplication of the pool
- ASSERT checks on every allocate and deallocate — catches bugs immediately
- class final — prevents inheritance from breaking pool invariants
- ⚠ NOT thread-safe — designed for single-thread use only (by design, not oversight)

3.
Lock-Free Queue

SPSC (Single Producer, Single Consumer) ring buffer for inter-thread communication

Files: lf_queue.h, lf_queue_example.cpp

Locks vs. Lock-Free — Why It Matters

Mutex (Lock-Based):

- Thread acquires lock before writing or reading
- If lock is held → thread is BLOCKED by OS
- Context switch: save/restore all registers + stack
- Cost: 1-10 μs per lock contention event
- Unpredictable: OS scheduler decides when to wake up

Lock-Free (SPSC Queue):

- No locks at all — only atomic variables
- Writer and reader never block each other
- No context switches, no OS involvement
- Cost: ~10-50 ns per enqueue/dequeue
- Deterministic: pure user-space operations

In trading systems, 1 μs of lock contention = losing the race to fill an order. Lock-free structures eliminate this entirely for the SPSC (single-producer, single-consumer) case.

LFQueue — Circular Buffer Design

```cpp
template<typename T>
class LFQueue final {
    std::vector<T> store_;            // ring buffer
    std::atomic<size_t> next_write_index_ = {0};
    std::atomic<size_t> next_read_index_  = {0};
    std::atomic<size_t> num_elements_     = {0};
};
```

Why std::atomic?

Producer updates write_index

Consumer reads write_index

→ Both threads access same variable

→ Must be atomic to avoid data races

Ring Buffer Mechanics:

next_index = (current_index + 1) % store_.size()

When index reaches the end of the array, it wraps back to 0 — like a circular track.

The queue is empty when read_index == write_index (and num_elements == 0).

The queue is full when the writer catches up to the reader after one full revolution.

LFQueue — Write & Read Operations

```cpp
// ===== PRODUCER (writer thread) =====
auto getNextToWriteTo() noexcept {
    return &store_[next_write_index_];
}
auto updateWriteIndex() noexcept {
    next_write_index_ =
        (next_write_index_ + 1) % store_.size();
    num_elements_++;
}

// ===== CONSUMER (reader thread) =====
auto getNextToRead() const noexcept -> const T* {
    return (next_read_index_ == next_write_index_)
        ? nullptr : &store_[next_read_index_];
}
auto updateReadIndex() noexcept {
    next_read_index_ =
        (next_read_index_ + 1) % store_.size();
    ASSERT(num_elements_ != 0, ...);
    num_elements_--;
}
```

Two-Step Design:

Writer:

1. Get pointer to write slot
2. Fill in data directly
3. Call updateWriteIndex()

Consumer sees data only AFTER step 3

Reader:

1. getNextToRead() → nullptr if empty
2. Process the data
3. Call updateReadIndex()

No locks, no copies — just pointer math

Why split into 2 steps?

Avoids race condition: data is fully written before index advances. Also avoids copying large objects — write in-place.

LFQueue — Producer-Consumer Example

```cpp
// Producer (main thread)
for (int i = 0; i < 50; ++i) {
    *(lfq.getNextToWriteTo()) = MyStruct{i, i*10, i*100};
    lfq.updateWriteIndex();
    sleep_for(1s);
}

// Consumer (background thread)
while (lfq.size()) {
    const auto d = lfq.getNextToRead();
    lfq.updateReadIndex();
    // process d...
}
```

Timeline:

→ Consumer sleeps 5s first

→ producer fills queue

→ consumer wakes up and starts draining

→ both run at 1 element/sec

→ pipeline parallelism!

	MemPool (4.2)	LFQueue (4.3)
Purpose	Fast object creation/destruction	Inter-thread data transfer
Thread safety	Single-thread only	Two threads (1 writer, 1 reader)
Needs atomic?	No	Yes — shared indices

4.
Low-Latency Logging Framework

Asynchronous logging that offloads disk I/O from the critical path

Files: time_utils.h, logging.h, logging_example.cpp

Synchronous vs. Asynchronous Logging

Synchronous Logging (Bad for Latency):

- Application thread formats the string itself
- Application thread writes to disk directly
- Disk I/O: 1-100 μs (unpredictable!)
- String formatting: 1-10 μs
- Total: your critical path is blocked for μs!

Asynchronous Logging (This Book's Approach):

- Critical thread only pushes raw data to LFQueue
- Cost: ~tens of nanoseconds (memory write only)
- Background logger thread reads from queue
- Logger thread does formatting + disk I/O
- Critical path is never blocked by I/O

Critical Thread ——push——> [LFQueue<LogElement>] ——pop——> Logger Thread ——> Disk File

LogElement — Union-Based Data Packing

```cpp
enum class LogType : int8_t {
    CHAR, INTEGER, LONG_INTEGER,
    LONG_LONG_INTEGER,
    UNSIGNED_INTEGER, UNSIGNED_LONG,
    UNSIGNED_LONG_LONG,
    FLOAT, DOUBLE
};

struct LogElement {
    LogType type_ = LogType::CHAR;
    union {
        char   c; int    i; long   l;
        long long ll; unsigned u;
        unsigned long ul;
        unsigned long long ull;
        float  f; double d;
    } u_;
};
```

Why union instead of std::variant?

union: all members share one memory slot. Size = largest member (8 bytes for double).

std::variant (C++17) adds type-safety overhead — book explicitly avoids it for performance.

LogElement total size ≈ 9-16 bytes (type tag + union + padding)

Logger Class — Architecture Overview

```
class Logger final {
    std::ofstream file_;                          // Output file
    LFQueue<LogElement> queue_;                   // 8M-element lock-free queue
    std::atomic<bool> running_ = {true};          // Controls logger thread
    std::thread* logger_thread_ = nullptr;        // Background thread

    Logger(const std::string& file_name)
      : file_(file_name), queue_(8 * 1024 * 1024) {
        logger_thread_ = createAndStartThread(-1, "Logger", [this](){flushQueue();});
        ASSERT(logger_thread_ != nullptr, "Failed to start Logger thread.");
    }
}
```

Constructor: Opens file, pre-allocates 8M-element queue, spawns background thread

flushQueue(): Logger thread's main loop — reads LogElements, writes to file via switch(type_)

Destructor: Sets running_=false, waits for queue to drain, joins thread, closes file — no data loss

Sleep on empty: Logger thread sleeps 1ms when queue is empty — saves CPU, acceptable latency for logs

log() — printf-Style Variadic Template

```cpp
template<typename T, typename... A>
auto log(const char* s, const T& value, A... args) noexcept {
    while (*s) {
        if (*s == '%') {
            if (*(s+1) == '%') { ++s; }  // escape: %% → literal %
            else {
                pushValue(value);           // push current arg to queue
                log(s + 1, args...);        // recurse with remaining args
                return;
            }
        }
        pushValue(*s++);  // push literal character
    }
}
```

Recursive template expansion: log("Price:% Qty:%\n", 100.5, 42) expands into:

→ push 'P','r','i','c','e',':' → push 100.5 → recurse

→ push ' ','Q','t','y',':' → push 42 → recurse

→ push '\n' → done (base case: no more args)

pushValue() has overloads for every type (char, int, double, string...) → each creates a LogElement and writes it to the LFQueue.

pushValue() — Type-Specific Queue Insertion

```cpp
// Core: write LogElement directly to LFQueue
auto pushValue(const LogElement &le) noexcept {
    *(queue_.getNextToWriteTo()) = le;
    queue_.updateWriteIndex();
}

// Single char → create LogElement, push
auto pushValue(const char value) noexcept {
    pushValue(LogElement{LogType::CHAR, {.c=value}});
}

// C-string → push each char one by one
auto pushValue(const char *value) noexcept {
    while (*value) { pushValue(*value); ++value; }
}

// Numeric types → tag + union
auto pushValue(const int v) noexcept {
    pushValue(LogElement{LogType::INTEGER, {.i=v}});
}
auto pushValue(const double v) noexcept {
    pushValue(LogElement{LogType::DOUBLE, {.d=v}});
}  // ... same pattern for long, float, unsigned,
etc.
```

```
Call Hierarchy:
pushValue(LogElement)
  └─ writes to LFQueue
      ▲
      ├─ pushValue(char)
      ├─ pushValue(int)
      ├─ pushValue(double)
      ├─ pushValue(float)
      ├─ pushValue(long)
      ├─ pushValue(unsigned)
      └─ ...

pushValue(const char*)
  └─ loops: pushValue(char)

pushValue(std::string)
  └─ calls pushValue(c_str())
```

5.
TCP/UDP Network Sockets

TCP for reliable order flow, UDP multicast for market data broadcasting

Files: socket_utils.h/cpp, tcp_socket.h/cpp, tcp_server.h/cpp, mcast_socket.h/cpp

TCP vs. UDP — Two Protocols, Two Use Cases

Feature	TCP	UDP (Multicast)
Reliability	Guaranteed delivery + ordering	Best effort — may lose packets
Connection	1-to-1 (point-to-point)	1-to-many (group broadcast)
Speed	Slower (ACKs, retransmission)	Faster (no overhead)
Use in Trading	Order entry & execution	Market data distribution
Book's Class	TCPSocket / TCPServer	McastSocket

Why UDP multicast for market data?

The exchange sends ONE packet and the network hardware replicates it to all subscribers. With TCP, the exchange would need a separate connection per client — O(N) work vs O(1).

Analogy: TCP = registered mail (confirmed delivery). UDP multicast = radio broadcast (tune in to receive).

Socket Configuration — Key Settings

Non-Blocking Mode	fcntl(fd, F_SETFL, O_NONBLOCK) — recv/send return immediately if no data; critical path never blocks
TCP_NODELAY	Disables Nagle algorithm — sends data immediately without waiting to batch small packets; essential for low latency
SO_TIMESTAMP	Kernel attaches a timestamp to each received packet — enables precise latency measurement
Multicast TTL	Controls how many router hops a multicast packet can traverse; prevents infinite circulation
Multicast Join	IP_ADD_MEMBERSHIP — subscribes the socket to a multicast group address (like joining a chat room)

TCPSocket — Send/Receive with Callbacks

```
struct TCPSocket {
    int fd_ = -1;                      // File descriptor (OS handle)
    char *snd_buffer_, *rcv_buffer_;  // 64MB pre-allocated buffers
    size_t next_rcv_valid_index_ = 0; // Tracks received data position
    size_t next_snd_valid_index_ = 0; // Tracks pending send data
    std::function<void(TCPSocket*, Nanos)> recv_callback_;
    Logger& logger_;

    auto connect(ip, iface, port, is_listening) → int;  // client or server
    auto send(const void* data, size_t len) → void;     // copy to snd_buffer
    auto sendAndRecv() → bool;                          // actual I/O via syscall
};
```

Design Highlights:

- Pre-allocated 64MB buffers — no malloc on the critical path
- Callback-driven: recv_callback_ fires when data arrives — event-driven architecture
- send() only copies to buffer; actual network I/O happens in sendAndRecv()
- SO_TIMESTAMP extracts kernel-level receive timestamp for latency measurement

TCPSocket — Send/Receive with Callbacks

```
struct TCPSocket {
    int fd_ = -1;                      // File descriptor (OS handle)
    char *snd_buffer_, *rcv_buffer_;  // 64MB pre-allocated buffers
    size_t next_rcv_valid_index_ = 0; // Tracks received data position
    size_t next_snd_valid_index_ = 0; // Tracks pending send data
    std::function<void(TCPSocket*, Nanos)> recv_callback_;
    Logger& logger_;

    auto connect(ip, iface, port, is_listening) → int;  // client or server
    auto send(const void* data, size_t len) → void;     // copy to snd_buffer
    auto sendAndRecv() → bool;                          // actual I/O via syscall
};
```

Design Highlights:

- Pre-allocated 64MB buffers — no malloc on the critical path
- Callback-driven: recv_callback_ fires when data arrives — event-driven architecture
- send() only copies to buffer; actual network I/O happens in sendAndRecv()
- SO_TIMESTAMP extracts kernel-level receive timestamp for latency measurement

TCPSocket::sendAndRecv() — Non-Blocking I/O

```cpp
// Simplified from tcp_socket.cpp
auto TCPSocket::sendAndRecv() noexcept -> bool {
  // 1. SEND pending data from snd_buffer_
  if (next_snd_valid_index_) {
    ssize_t n = ::send(fd_, snd_buffer_,
                       next_snd_valid_index_,
                       MSG_DONTWAIT | MSG_NOSIGNAL);
    // MSG_DONTWAIT = non-blocking
    // MSG_NOSIGNAL = no SIGPIPE on disconnect
    if (n > 0) next_snd_valid_index_ -= n;
  }

  // 2. RECV into rcv_buffer_
  ssize_t n = ::recv(fd_, rcv_buffer_ +
                     next_rcv_valid_index_,
                     RCV_BUF_SIZE - next_rcv_valid_index_,
                     MSG_DONTWAIT);
  if (n > 0) {
    next_rcv_valid_index_ += n;
    recv_callback_(this, rx_time);
    // ^ fire callback with kernel timestamp
  }
  return (next_rcv_valid_index_ > 0);
}
```

Why This Matters:

Non-blocking

MSG_DONTWAIT: send/recv return immediately even if no data — critical path never stalls

Pre-allocated buffers

64MB snd_buffer_ and rcv_buffer_ allocated once at construction — zero malloc

Deferred I/O

send() copies to buffer; actual syscall only happens here in sendAndRecv()

Callback-driven

recv_callback_ fires when data arrives — event-driven architecture

TCPServer — Managing Connections with epoll

epoll is Linux's high-performance I/O event notification system. Instead of polling every socket, it only notifies about sockets that have pending events — O(active) not O(total).

```
TCPServer::poll() {
  // 1. Clean up disconnected sockets
  // 2. epoll_wait() → get active events
  // 3. Classify each event:
  //    - EPOLLIN on listener → new connection
  //    - EPOLLIN on client  → data to read
  //    - EPOLLOUT           → ready to write
  //    - EPOLLERR/EPOLLHUP  → disconnected
  // 4. accept() new connections, add to epoll
}
```

epoll in 3 steps:

1. epoll_create() → create instance
2. epoll_ctl(ADD) → register sockets
3. epoll_wait() → block until events

Edge-triggered (EPOLLET) mode is used — only notifies on state change, not level.

McastSocket — UDP Multicast for Market Data

McastSocket wraps UDP multicast communication. Like TCPSocket, it has pre-allocated buffers, send/recv methods, and a callback for received data. But it has no connection management.

Method	Purpose
init()	Creates UDP socket, sets non-blocking, configures TTL
join(group_ip)	Subscribes to a multicast group — like joining a chat room
send(data, len)	Copies data to send buffer (identical pattern to TCPSocket)
sendAndRecv()	Performs actual UDP sendto/recvfrom system calls
leave()	Unsubscribes from the multicast group — stops receiving

Summary & Wrap-Up

**How the five building blocks fit together
and prepare us for the trading system implementation**

Summary

1. Thread Utilities
   - Create threads, pin to CPU cores via affinity
2. Memory Pool
   - Pre-allocate at init → zero malloc on hot path
3. Lock-Free Queue
   - SPSC FIFO with std::atomic — no locks, no context switches
4. Low-Latency Logging Framework
   - Async logging: push to LFQueue (ns) → background thread writes disk
5. TCP/UDP Network Sockets
   - Non-blocking TCP/UDP + epoll; TCPServer, TCPSocket, McastSocket

Questions & Discussions

Next week: Trading Ecosystem Design

- ***trading system requirements and constraints***
- ***overall architecture (exchange, clients, strategy engine)***